SAKARYA ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

NESNE YÖNELİMLİ ANALİZ VE TASARIM DERSİ
1. PROJE ÖDEVİ

KAYNAK KODLAR RAPORU

"Akıllı Tedarik ve Lojistik Yönetim Sistemi"

Öğrenci Adı Soyadı: Zeynep Uludağ

Numarası: B241210029

1. Proje Bilgileri:

Bu projede bir e-ticaret lojistik merkezinin ürün, stok, sipariş, ödeme, kargo ve bildirim işlemlerini yönetebilen basit bir sistem geliştirilmiştir. Proje C# dili ile yazılmıştır. Kod yapısı MVC Model-View-Controller mimarisine uygun olarak klasörlere ayrılmıştır.

Projede amaç nesneye dayalı programlama mantığını kullanarak daha düzenli ve okunabilir bir program geliştirmektir. Bu yüzden kodlarda farklı tasarım desenleri kullanılmıştır.

Program çalıştırıldığında sırasıyla; ürünler listelenir, sipariş oluşturulur, ödeme yapılır, sipariş durumları değiştirilir, kargo bilgisi oluşturulur, stok kontrolü yapılır ve testler çalıştırılır.

Kaynak kodlar ayrı bir klasör olarak teslim edilmiştir. Bu raporda program dosyaları ve klasör yapısının açıklamaları bulunmaktadır.

2. Proje Klasör Yapısı

Projede kodların daha düzenli olması için sınıflar görevlerine göre farklı klasörlere ayrılmıştır:

Program.cs : Programın buradan başlar. Demo senaryosu burada çalışır.

SmartSupplyLogistics.csproj : C# proje dosyasıdır. Projenin .NET sürümünü belirtir.

Models: Ürün, sipariş ve kullanıcı gibi temel sınıflar burada bulunur.

Controllers: Programdaki iş akışlarını yöneten sınıflar burada bulunur.

Views: Konsola çıktı veren sınıflar burada bulunur.

Patterns: Tasarım desenleri ile ilgili sınıflar burada bulunur.

Tests: Basit unit test metotları burada bulunur.

3. Model-View-Controller Açıklaması

Model katmanında sistemde kullanılan ana nesneler vardır. Örneğin ürün, sipariş, kullanıcı ve sipariş durumları bu katmanda yer almaktadır. “Models” klasöründe ürünlerle, siparişlerle ve kullanıcılarla ilgili sınıflar bulunmaktadır. View katmanı kullanıcıya gösterilecek çıktıları hazırlar. Bu proje web uygulaması olmadığı için View sınıfları HTML sayfası üretmez. Bunun yerine konsola yazı yazdırır. Örneğin ürün listesini, sipariş bilgisini ve stok bilgilerini konsola yazdıran sınıflar “Views” klasörğndedir. Controller katmanı, Model ve View arasında bağlantı kurar. Kullanıcının yapmak istediği işlemleri yönetir. Örneğin sipariş oluşturma, ödeme alma, ürün listeleme ve stok güncelleme işlemleri Controller sınıfları ile yapılır.

4. Algoritma ve Projenin Genel Çalışma Şekli

1- Logger nesnesi hazırlanır.
2- Ürün, sipariş ve stok controller nesneleri oluşturulur.
3- Kullanıcılar oluşturulur.
4- Basit ürünler oluşturulur.

5- Bileşik ürün oluşturulur.
6- Ürünler ekrana yazdırılır.
7- Sipariş oluşturulur.
8- Ödeme yapılır.
9- Sipariş durumları değiştirilir.
10- Kargo firması seçilir.
11- Kargo takip numarası ve kargo ücreti oluşturulur.
12- Stok kontrolü yapılır.
13- Kritik stok varsa bildirim gönderilir.
14- İade senaryosu çalıştırılır.
15- Testler çalıştırılır.

- **Models/Products Klasörü:**

Bu klasörde ürünlerle ilgili sınıflar bulunmaktadır. IProduct.cs dosyasında ürünlerin ortak özelliklerini belirleyen bir interface vardır. Bu interface sayesinde basit ürünler ve bileşik ürünler aynı türden nesneler gibi kullanılabilir. Bu da kodun daha esnek olmasını sağlar. Ürünlerin ortak olarak sahip olması gereken bazı metotlar şunlardır:

GetName()

GetPrice()

GetStock()

GetDescription()

SimpleProduct.cs dosyasında SimpleProduct, sistemdeki normal ürünleri temsil eder. Örneğin kalem, RAM veya CPU gibi tek başına satılabilen ürünler bu sınıf ile oluşturulur. Bu sınıfta ürünün;

adı

fiyatı

stok miktarı

kategorisi tutulur.

Örneğin program çıktısında görülen şu ürünler SimpleProduct sınıfı ile oluşturulmuştur:

Kalem

RAM

CPU

CompositeProduct.cs dosyasında CompositeProduct, içinde başka ürünler bulunan bileşik ürünleri temsil eder.

Örneğin bilgisayar kasası tek başına bir ürün gibi görünür ama içinde RAM ve CPU gibi parçalar bulunabilir. Bu durumda bilgisayar kasası CompositeProduct olarak modellenmiştir. Bu sınıfta Composite Pattern kullanılmıştır. Bileşik ürünün fiyatı, içindeki parçaların fiyatları ve montaj ücreti kullanılarak hesaplanır. Stok miktarı ise içindeki parçalardan en az stoğa sahip olan ürüne göre belirlenir. Örneğin RAM stoğu 3, CPU stoğu 2 ise bilgisayar kasasının üretilebilir stoğu 2 olur. Çünkü CPU stoğu daha azdır.

Category.cs dosyasındaki Category sınıfı ürün kategorilerini temsil eder. Örneğin:

Elektronik
Kırtasiye
Bilgisayar Parçaları gibi kategoriler bu sınıf ile tutulabilir.

ProductFactory.cs dosyasındaki ProductFactory, ürün nesnelerini oluşturmak için kullanılır. Normalde her yerde new SimpleProduct() veya new CompositeProduct() yazılabilir. Ancak bu durum kodun farklı yerlerine dağılırsa yönetim zorlaşır. Bu yüzden ürün oluşturma işlemi factory sınıfı içine alınmıştır. Bu sınıfta Factory Method mantığı kullanılmıştır. Böylece ürün oluşturma işlemi tek bir yerden kontrol edilir.

- **Models/Orders Klasörü:**

Bu klasörde siparişlerle ilgili sınıflar bulunur. Order.cs dosyasındaki Order sınıfı, sistemdeki siparişleri temsil eder.

sipariş numarası

kullanıcı bilgisi

sipariş ürünleri

toplam tutar

sipariş durumu

kargo takip numarası

kargo firması

bir siparişin bu bilgileri tutulur. Sipariş ilk oluşturulduğunda durumu Beklemede olur. Daha sonra State Pattern ile durum değiştirilir. Sipariş durumları sırayla şu şekilde ilerler:

Beklemede -> Onaylandı -> Hazırlanıyor -> Kargoda -> Teslim Edildi

Eğer ürün kargoda hasarlı olursa sipariş iade durumuna geçebilir.

OrderItem.cs dosyasındaki OrderItem, siparişin içindeki ürünleri ve adet bilgisini tutar.

Bir sipariş birden fazla üründen oluşabilir. Her ürün için ürün nesnesi ve kaç adet alındığı bilgisi bu sınıfta tutulur. Örneğin bir müşteri 1 adet bilgisayar kasası ve 2 adet kalem sipariş ederse bunlar ayrı OrderItem olarak eklenir.

- **Models/Users Klasörü:**

Bu klasörde kullanıcı ve rol sınıfları vardır. Users.cs dosyasındaki User sınıfı sistemdeki kullanıcıları temsil eder. Kullanıcı bilgileri şunlardır:

kullanıcı id

ad

e-posta

rol

Kullanıcının hangi işlemleri yapabileceği rolüne göre belirlenir. IUserRole**.cs** interface'i kullanıcı rollerinin ortak yapısını belirtir. Her rolün adı ve yetkileri farklı olabilir. Bu yüzden roller ayrı sınıflar olarak yazılmıştır.

AdminRole.cs : Admin sistem günlüklerini inceleyebilir, kullanıcı yetkilendirebilir ve genel sistem işlemlerini yapabilir.

PersonnelRole.cs : PersonnelRole, depo personeli rolünü temsil eder. Depo personeli stok güncelleme ve ürün montajı gibi işlemleri yapabilir.

CustomerRole**.cs :** CustomerRole, müşteri rolünü temsil eder. Müşteri ürünleri listeleyebilir, sipariş verebilir ve ödeme yapabilir.

CourierRole**.cs :** CourierRole, kurye rolünü temsil eder. Kurye sipariş durumunu güncelleyebilir ve kargo süreciyle ilgili işlemleri yapabilir.

- **Models/Controllers Klasörü:**

Controller sınıfları programdaki iş akışlarını yönetir. View ve Model sınıfları arasında bağlantı kurar. ProductController, ürünlerle ilgili işlemleri yönetir. Bu sınıfın görevleri:

ürün oluşturmak

ürünleri listelemek

ürünleri view sınıfına göndermek

Ürün oluşturma işlemlerinde ProductFactory sınıfından yararlanılır.

OrderController, siparişle ilgili işlemleri yönetir. Görevleri:

sipariş oluşturmak

ödeme işlemini başlatmak

sipariş durumunu ilerletmek

kargo bilgisi eklemek

iade sürecini başlatmak

Ödeme işlemi için Strategy Pattern kullanılır. Sipariş durumu değiştirme işlemi için State Pattern kullanılır. Kargo işlemleri için Adapter ve Decorator sınıfları kullanılır.

StockController, stok işlemlerini yönetir. Bu sınıf stok güncelleme işlemlerini yapar. Stok kritik seviyeye düşerse StockManager üzerinden bildirim sistemi çalıştırılır. Stok değişimleri ayrıca Logger sınıfı ile loglanır.

- **Models/Views Klasörü:**

View sınıfları konsola çıktı basmak için kullanılır. Bu proje bir web uygulaması olmadığı için View sınıfları sayfa üretmez. Bunun yerine kullanıcıya konsol ekranında bilgi gösterir.

ProductView.cs : Ürün bilgilerini konsola yazdırır.

OrderView.cs : Sipariş bilgilerini konsola yazdırır. Sipariş numarası, sipariş durumu ve tutar bilgileri bu sınıf ile gösterilir. Ayrıca kargo takip numarası ve kargo ücreti de ekrana yazdırılır.

StockView.cs : Stok güncelleme ve bildirim bilgilerini konsola yazdırır. Stok kritik seviyeye düştüğünde kullanıcıya bilgi verir.

UserView.cs : Kullanıcı bilgilerini konsola yazdırmak için hazırlanmıştır. Kullanıcının adı, e-posta adresi ve rolü bu sınıf ile gösterilebilir.

- **Models/Patterns Klasörü:**

Bu klasör projedeki tasarım desenlerini içeren kod dosyalarını bulundurur. Tasarım desenlerinin neden seçildiği ve hangi problemi çözdüğü Tasarım Raporu'nda daha detaylı anlatılmıştır. Bu raporda ise sadece bu klasördeki dosyaların programda ne işe yaradığı açıklanmıştır.

Patterns klasörü içinde ödeme işlemleri, stok bildirimi, kargo API uyarlaması, loglama, ek kargo hizmetleri ve lojistik işlemleri ile ilgili sınıflar vardır. Bu sınıflar programın daha düzenli ve geliştirilebilir olması için kullanılmıştır.

Patterns/Strategy Klasörü:

Bu klasör ödeme işlemleri ile ilgili sınıfları içerir.

IPaymentStrategy.cs dosyasında ödeme yöntemlerinin ortak yapısı tanımlanmıştır. Kredi kartı, havale ve kripto ödeme gibi farklı ödeme yöntemleri bu interface üzerinden çalışır.

CreditCardStrategy.cs kredi kartı ile ödeme işlemini temsil eder. Programın demo çıktısında ödeme bu sınıf üzerinden yapılmıştır.

TransferStrategy.cs havale ile ödeme işlemini temsil eder. Bu sınıf programda farklı ödeme yöntemlerinin desteklenebileceğini göstermek için eklenmiştir.

CryptoStrategy.cs kripto ödeme yöntemini temsil eder. Ödev dosyasında ileride kripto ödeme gibi yöntemlerin eklenebilir olması istendiği için bu sınıf projeye eklenmiştir.

PaymentManager.cs ise seçilen ödeme yöntemini çalıştıran sınıftır. Yani ödeme işlemi doğrudan tek bir ödeme türüne bağlı değildir. Hangi ödeme sınıfı verilirse ona göre işlem yapılır.

Patterns/Observer Klasörü:

Bu klasör stok bildirimi ile ilgili sınıfları içerir.

IStockObserver.cs dosyasında stok bildirimi alacak sınıfların ortak yapısı tanımlanmıştır.

EmailNotifier.cs stok kritik seviyeye düştüğünde satın alma birimine e-posta gönderilmiş gibi konsola çıktı verir.

SystemNotifier.cs depo sorumlusuna sistem içi bildirim gönderilmiş gibi konsola çıktı verir.

StockManager.cs stok seviyesini kontrol eden sınıftır. Eğer ürün stoğu belirlenen eşik değerin altına düşerse kayıtlı bildirim sınıflarını çalıştırır. Program çıktısında RAM ve CPU stokları kritik seviyede olduğu için e-posta ve sistem bildirimi konsola yazdırılmıştır.

Patterns/Singleton Klasörü:

Bu klasörde loglama işlemleri için kullanılan Logger.cs dosyası vardır.

Logger.cs, sistemde yapılan kritik işlemleri kaydetmek için kullanılır. Örneğin sipariş oluşturulduğunda, ödeme onaylandığında, sipariş durumu değiştiğinde veya stok güncellendiğinde log kaydı oluşturulur. Program çalışırken loglar hem konsola yazdırılır hem de system-log.txt dosyasına kaydedilir. Böylece sistemde hangi işlemin ne zaman yapıldığı takip edilebilir.

Patterns/Adapter Klasörü:

Bu klasör kargo firmaları ile ilgili sınıfları içerir.

Ödev senaryosunda Aras, Yurtiçi ve GlobalExpress gibi farklı kargo firmalarının farklı API yapılarına sahip olduğu belirtilmiştir. Bu projede bu firmalar için ayrı sınıflar yazılmıştır.

ILogisticsProvider.cs dosyası sistemin kullandığı ortak kargo arayüzüdür.

ArasCargoApi.cs, YurticiCargoApi.cs ve GlobalExpressApi.cs dosyaları farklı kargo firmalarının API yapısını temsil eder.

ArasAdapter.cs, YurticiAdapter.cs ve GlobalExpressAdapter.cs dosyaları ise bu farklı API yapılarını sistemin anlayacağı ortak yapıya çevirir. Bu sayede programda kargo firması değişse bile sistem aynı mantıkla çalışmaya devam eder.

Patterns/Decorator Klasörü:

Bu klasör kargoya ek hizmet eklemek için kullanılır. LogisticsDecorator.cs temel decorator sınıfıdır.

InsuredShippingDecorator.cs sigortalı gönderim hizmetini ekler.

FragileItemDecorator.cs kırılacak eşya koruması hizmetini ekler.

Program çıktısında kargo firması şu şekilde görünmektedir:

Aras Kargo + Sigortalı Gönderim + Kırılacak Eşya Koruması

Bu çıktı, normal kargo hizmetinin üzerine sigortalı gönderim ve kırılacak eşya koruması eklendiğini göstermektedir.

Patterns/Facade Klasörü:

Bu klasörde LogisticsFacade.cs dosyası bulunur.

Bu sınıf kargo işlemlerini daha sade hale getirmek için hazırlanmıştır. Kargo ücreti hesaplama, takip numarası üretme ve loglama gibi işlemler normalde ayrı ayrı yapılabilir. Ancak bu işlemleri tek bir sınıf üzerinden yönetmek kodun daha okunabilir olmasını sağlar.

• **Tests Klasörü:**

Bu klasör programdaki basit unit testleri içerir.

ProjectTests.cs dosyasında kritik metotların doğru çalışıp çalışmadığını kontrol eden testler yazılmıştır. Bu testler program çalıştırıldığında otomatik olarak çalışır.

Daha detaylı açıklaması test raporunda mevcuttur.

5. Program Çıktısının Açıklaması

Program çalıştığında ilk olarak ürün listesi ekrana yazdırılır. Örnek olarak basit ürünler ve bileşik ürünler gösterilir. Kalem, RAM ve CPU basit ürünlerdir. Bilgisayar Kasası ise RAM ve CPU parçalarından oluşan bileşik üründür.

Daha sonra bir sipariş oluşturulur. Sipariş ilk oluşturulduğunda durumu Beklemede olur.

Ödeme işlemi başarılı olduktan sonra sipariş durumu değişmeye başlar. Sipariş önce Onaylandı, sonra Hazırlanıyor, daha sonra Kargoda durumuna geçer.

Kargo işlemi sırasında kargo firması seçilir, takip numarası üretilir ve kargo ücreti hesaplanır. Programda örnek olarak Aras Kargo kullanılmıştır. Ayrıca sigortalı gönderim ve kırılacak eşya koruması da ek hizmet olarak eklenmiştir.

Stok kontrolü kısmında RAM ve CPU stokları kritik seviyede olduğu için sistem bildirim gönderir. Satın alma birimine e-posta gönderilmiş gibi çıktı verilir. Depo sorumlusuna da sistem içi bildirim gönderilmiş gibi çıktı verilir.

En son testler çalıştırılır ve bütün testlerin geçtiği ekrana yazdırılır.

6. Kaynak Kodlarda Dikkat Edilen Noktalar

Kodlar yazılırken sınıfların görevlerine göre ayrılmasına dikkat edilmiştir. Her sınıfın mümkün olduğunca tek bir görevi olması hedeflenmiştir.

Örneğin ürünlerle ilgili kodlar Models/Products klasöründe, siparişlerle ilgili kodlar Models/Orders klasöründe, ödeme ve kargo gibi özel yapılar ise Patterns klasöründe tutulmuştur.

Controller sınıfları iş akışını yönetmek için kullanılmıştır. View sınıfları ise sadece konsola çıktı vermek için kullanılmıştır. Böylece kodlar birbirinden daha düzenli şekilde ayrılmıştır.

Kodlarda interface yapıları kullanılmıştır. Interface kullanılması sayesinde farklı ödeme yöntemleri, farklı kargo firmaları ve farklı bildirim türleri aynı sistem içinde daha kolay kullanılabilmiştir. Programda gerçek veritabanı, gerçek e-posta gönderme sistemi veya gerçek kargo API bağlantısı kullanılmamıştır. Bu işlemler ödev kapsamında konsol çıktısı ile simüle edilmiştir. Ancak sınıf yapısı ileride gerçek sistemlere bağlanabilecek şekilde düzenlenmiştir.

Not: Programın çalışması için gerekli kaynak kodlar ayrı klasör halinde teslim edilmiştir. Bu raporda da kaynak kodların genel yapısı ve dosyaların görevleri açıklanmıştır.